

C# - Phần 9.10: Bài tổng hợp Delegate, Lambda và Event

Capstone Phần 9 - Student Course Registration Manager

Mục tiêu: Tự kết hợp Delegate, Multicast Delegate, Action, Func, Predicate, Lambda, Event, Publisher/Subscriber, List<T>, Validation và Exception trong một chương trình C# hoàn chỉnh.

1. Bài toán

Xây dựng chương trình Console quản lý việc sinh viên đăng ký môn học.

Luồng tổng quát:

```
Sinh viên đăng ký môn
    ↓
RegistrationService xử lý
    ↓
kiểm tra dữ liệu
    ↓
tìm sinh viên / môn học
    ↓
đăng ký thành công
    ↓
phát Event
    ↓
EmailService
LogService
NotificationService
nhận thông báo
```

2. Kiến thức phải sử dụng

- Delegate.
- Multicast Delegate.
- Action.
- Func.
- Predicate.
- Lambda Expression.
- Event.
- Publisher / Subscriber.
- List<T>.
- Validation + Exception cơ bản.

3. Cấu trúc project đề xuất

```
StudentCourseRegistration/
|
|— Models/
|   |— Student.cs
|   |— Course.cs
|   |— Registration.cs
|
|— Services/
|   |— RegistrationService.cs
|   |— EmailService.cs
```

```

|   |   | LogService.cs
|   |   | NotificationService.cs
|   |   |
|   |   | Delegates/
|   |   |   | RegistrationDelegate.cs
|   |   |
|   |   | Program.cs

```

Không bắt buộc phải đúng folder như trên, nhưng nên tách class riêng để dễ đọc.

4. Model Student

```

public string Id { get; }
public string FullName { get; }

```

Constructor phải validation:

- Id không null / empty / whitespace rồi Trim.
- FullName không null / empty / whitespace rồi Trim.

Có thể override ToString() để tiện in thông tin.

5. Model Course

```

public string Id { get; }

public string Name { get; }

public int Credits { get; }

public decimal FeePerCredit { get; }

```

Validation:

- Id / Name không null / empty / whitespace.
- Credits > 0.
- FeePerCredit > 0.

Ví dụ dữ liệu:

```

C01 - C# Programming - 3 credits
C02 - Database - 4 credits
C03 - Web Development - 3 credits

```

6. Model Registration

Đại diện cho một Student đã đăng ký một Course.

```

public Student Student { get; }

public Course Course { get; }

public DateTime RegisteredAt { get; }

public Registration(
    Student student,
    Course course)
{
    ...
    RegisteredAt = DateTime.Now;
}

```

7. RegistrationService - Publisher chính

Class này quản lý dữ liệu và nghiệp vụ đăng ký.

```
private readonly List<Student> students;
private readonly List<Course> courses;
private readonly List<Registration> registrations;
public RegistrationService()
{
    students = new List<Student>();
    courses = new List<Course>();
    registrations = new List<Registration>();
}
```

8. Các method cơ bản

```
public void AddStudent(Student student)

public void AddCourse(Course course)

public void RegisterCourse(
    string studentId,
    string courseId)
```

RegisterCourse() là nghiệp vụ chính của bài.

9. Predicate<T> và Lambda để tìm dữ liệu

Tìm Student theo ID có thể viết:

```
Predicate<Student> condition =
    student => student.Id == studentId;

Student? student =
    students.Find(condition);
```

Hoặc viết Lambda trực tiếp:

```
Student? student =
    students.Find(
        student => student.Id == studentId);
```

Tương tự với Course:

```
Course? course =
    courses.Find(
        course => course.Id == courseId);
```

Kiểu thức ghép: Predicate<T> + Lambda + List<T>.Find().

10. Kiểm tra đăng ký trùng

Trước khi tạo Registration, phải kiểm tra Student đã đăng ký Course đó chưa.

Dùng registrations.Exists(...) với Lambda. Điều kiện phải đồng thời kiểm tra Student.Id và Course.Id.

```
Registration
↓
Student.Id == studentId
AND
Course.Id == courseId
```

Nếu đã tồn tại:

```
throw new InvalidOperationException(...);
```

11. Func - tính học phí

Sử dụng:

```
Func<Course, decimal>
```

với Lambda để tính:

```
Credits × FeePerCredit
```

Ví dụ:

```
3 credits × 500000  
= 1,500,000
```

Bạn phải tự hoàn thiện Lambda.

12. Action - hiển thị Registration

Tạo Action<Registration> dùng Statement Lambda để in nhiều dòng.

```
Action<Registration> displayRegistration =  
    registration =>  
    {  
        ...  
    };
```

Ví dụ nội dung cần hiển thị:

```
Student: Nguyen Van A  
Course: C# Programming  
Credits: 3  
Registered At: ...
```

13. Custom Delegate và Multicast Delegate

```
public delegate void RegistrationMessageDelegate(  
    string message);
```

Tạo các method phù hợp, ví dụ:

```
ShowSuccessMessage(string message)
```

```
WriteConsoleMessage(string message)
```

Trong Program, tạo Multicast Delegate:

```
RegistrationMessageDelegate message = ...;  
  
message += ...;  
  
message("Registration completed");
```

Mục tiêu: Ôn lại Custom Delegate + Multicast Delegate + +=.

14. Event - phần quan trọng nhất

RegistrationService là Publisher.

```
public event Action<Registration>?  
    CourseRegistered;
```

Action<Registration> có chữ ký Registration -> void, nên Event Handler của Subscriber phải nhận Registration và trả void.

15. Phát Event

Chỉ phát Event sau khi toàn bộ nghiệp vụ đăng ký đã thành công:

```
tìm Student      ✓  
tìm Course       ✓  
kiểm tra trùng   ✓  
tạo Registration ✓  
Add vào List     ✓  
CourseRegistered?.Invoke(registration);
```

Lưu ý: Không phát Event trước khi Registration được tạo và thêm thành công.

16. Các Subscriber

```
EmailService  
→ SendRegistrationEmail(Registration registration)  
  
LogService  
→ WriteRegistrationLog(Registration registration)  
  
NotificationService  
→ ShowRegistrationNotification(Registration registration)
```

Cả ba handler đều có dạng Registration -> void nên phù hợp với Action<Registration> của Event.

17. Publisher / Subscriber trong bài

```
RegistrationService  
→ Publisher  
  
CourseRegistered  
→ Event  
  
EmailService  
→ Subscriber  
  
SendRegistrationEmail()  
→ Event Handler  
  
LogService  
→ Subscriber  
  
WriteRegistrationLog()  
→ Event Handler  
  
NotificationService  
→ Subscriber  
  
ShowRegistrationNotification()  
→ Event Handler
```

RegistrationService không được gọi trực tiếp các method của Subscriber. Nó chỉ phát Event:

```
CourseRegistered?.Invoke(registration);
```

18. Lũồng RegisterCourse()

```
RegisterCourse(studentId, courseId)
    ↓
validate studentId
    ↓
validate courseId
    ↓
tìm Student
    ↓
không có?
→ throw exception
    ↓
tìm Course
    ↓
không có?
→ throw exception
    ↓
kiểm tra đăng ký trùng
    ↓
đã đăng ký?
→ throw exception
    ↓
tạo Registration
    ↓
registrations.Add(...)
    ↓
CourseRegistered?.Invoke(...)
    ↓
EmailService
LogService
NotificationService
```

19. Dữ liệu test trong Program

```
Student student1 =
    new Student("SV01", "Nguyen Van A");

Student student2 =
    new Student("SV02", "Tran Thi B");

Course course1 =
    new Course(
        "C01",
        "C# Programming",
        3,
        500000);

Course course2 =
    new Course(
        "C02",
        "Database",
        4,
        450000);

Course course3 =
    new Course(
        "C03",
```

```
"Web Development",  
3,  
600000);
```

20. Yêu cầu Program.cs

1. Tạo RegistrationService.
2. Tạo EmailService, LogService, NotificationService.
3. Đăng ký SendRegistrationEmail và WriteRegistrationLog.
4. Thêm 2 Student.
5. Thêm 3 Course.
6. RegisterCourse("SV01", "C01").
7. Đăng ký thêm ShowRegistrationNotification.
8. RegisterCourse("SV02", "C02").
9. Hủy SendRegistrationEmail.
10. RegisterCourse("SV01", "C03").
11. Thử đăng ký trùng RegisterCourse("SV01", "C01").
12. Thử Student không tồn tại.
13. Thử Course không tồn tại.
14. Dùng Func<Course, decimal> để tính học phí.
15. Dùng Action<Registration> để hiển thị Registration.
16. Tạo Custom Delegate và thử Multicast Delegate.

Các test lỗi nên đặt trong các try/catch riêng để một exception không làm các test phía sau bị bỏ qua.

21. Thang điểm

Nội dung	Điểm
Model + validation	10
List<T> và quản lý dữ liệu	10
Custom Delegate	10
Multicast Delegate	5
Action	10
Func	10
Predicate	10
Lambda	10
Event	10
Publisher / Subscriber	10
Exception + test	5
Tổng	100

22. Câu hỏi lý thuyết cuối bài

1. Delegate trong C# là gì?
2. Custom Delegate khác Action / Func / Predicate như thế nào?
3. Vì sao Action không có return value?
4. Trong Func<Course, decimal>, Course và decimal lần lượt có vai trò gì?
5. Predicate<Student> có chữ ký như thế nào?

6. Lambda student => student.Id == studentId nhận kiểu gì và trả kiểu gì?
7. Vì sao Find() có thể nhận Lambda trên?
8. Multicast Delegate là gì?
9. Event được xây dựng dựa trên kiến thức nào?
10. += và -= có tác dụng gì với Event?
11. ?.Invoke() có nghĩa là gì?
12. RegistrationService là Publisher vì sao?
13. EmailService là Subscriber vì sao?
14. SendRegistrationEmail() là Subscriber hay Event Handler?
15. Vì sao RegistrationService không cần biết EmailService tồn tại?
16. Nếu thêm SmsService, có cần sửa RegistrationService không? Vì sao?
17. Event và Publisher / Subscriber có phải cùng một khái niệm không?
18. Vì sao Event chỉ nên được phát sau khi Registration được tạo thành công?

23. Phạm vi bài tập

Không yêu cầu kiến trúc nâng cao như:

- Clean Architecture.
- Repository Pattern.
- Dependency Injection.
- SOLID nâng cao.

Mục tiêu là tự kết hợp:

```
Delegate  
→ Action / Func / Predicate  
→ Lambda  
→ Event  
→ Publisher / Subscriber
```

Cách làm đề xuất: Làm Student, Course, Registration trước; sau đó RegistrationService; cuối cùng nối Event và các Subscriber. Khi hoàn thành bài này, bạn sẽ kết thúc Phần 9 và chuyển sang Phần 10 - LINQ.